

The New Quill: Tablas de procesos y conductos

Especificaciones para implementación.

Breve nomenclatura

Término	Definición
SLCONT	Contenedor que almacena una o varias SL y sus subls
SL	Sentencia Lógica: unidad mínima de input del jugador traducida a tokens o identificadores internos
TOKEN	En el source es una etiqueta que referencia a una palabra, item, npc, etc. Internamente se traduce a un identificador numérico.
SUBSL	SL anidada
NPC	<i>Non Player Character</i> : Personaje no jugador
ITEM	Objeto
PROCESS	Proceso dentro de una tabla, tiene un header y un body, contiene conductos
CONDUCT	Conducto: Una condición o una acción dentro de un proceso
BINDING	Asociación entre un nombre simbólico (ITEM , NPC) y la entidad concreta resuelta por el parser. Es visible en el proceso que casa la SL y en sus sub-procesos.
TABLE	Tabla de procesos, son fijas y tienen un lugar en el flujo del parser.

Tablas de procesos

Las tablas disponibles y su orden de ejecución:

Orden	Tabla	Ejecución
0	init	Cuando arranca la aventura, o se reinicia
1	location	Cuando cambia de ubicación
2	turn	Tras el input, antes de item
3	item	Tras el input, sólo si la SL contiene un item
4	npc	Tras el input, sólo si la SL contiene un NPC
5	response	Tras el input, en último lugar
6	cron	Por tiempo o turnos, procesos independientes

No se pueden crear otras tablas. Se admiten **INCLUDE** para repartir el contenido entre varios ficheros.

Input

Tras el input del jugador, el ciclo de tablas se ejecuta para cada SL empezando por la tabla 2 (**turn**), y continuando con la 3, 4 y 5 según apliquen. Al completar el ciclo, si quedan SUBSLs pendientes del bloque entrecomillado, se re-entra en el ciclo **sin turn** con la misma SL y la siguiente SUBSL. Si no quedan SUBSLs, se pasa a la siguiente SL principal (con **turn**). La tabla **turn** se ejecuta **una sola vez por SL principal**.

Ver *Secuencia Lógica* más adelante.

Cada tabla contiene una serie de procesos con expresiones de entrada. Como se ha mencionado, dichas tablas son fijas y se ejecutan en un momento predeterminado dentro del flujo del juego.

Terminadores

Los terminadores controlan el flujo de ejecución de las tablas:

Action	Description
DONE	Termina el ciclo de tablas para la SL/SUBSL actual. Si quedan SUBSLs, re-entra con la siguiente. Si no, pasa a la siguiente SL.
OK	Igual que DONE, pero imprime el mensaje <code>ok</code> antes de continuar.
NOOK	Imprime el mensaje <code>nook</code> y descarta todas las SUBSLs y SLs restantes. Devuelve el control al jugador.
END	Termina solamente la tabla actual y continúa con la siguiente tabla del ciclo.

Caer al final del cuerpo de un proceso sin terminador explícito equivale a un **END** implícito: la tabla se da por terminada, se continúa con la siguiente tabla del ciclo. Al agotar el ciclo completo de tablas sin **DONE**, **OK** ni **NOOK**, se aplica la misma lógica que **DONE**.

SL: Sentencia Lógica

Una **Sentencia Lógica** (SL) es la unidad mínima que el parser extrae del input del jugador. Tiene tres campos posicionales más un contenido opcional entrecomillado:

Formato SL: `[verb] [adverb] [article] <noun|pronoun> [adjective] [adverb] "<NPC SUBSL> [conjunction,punctuation]..." [conjunction,punctuation]...`

La SUBSL es una cadena opcional entrecomillada que contiene una o varias SL anidadas.

A partir del **noun** o del **pronoun**, el parser **infiere bindings**:

- Si el sustantivo resuelve a un Item declarado, **ITEM** queda ligado a ese Item.
- Si el sustantivo resuelve a un NPC declarado, **NPC** queda ligado a ese Character.

Estos bindings son visibles dentro del cuerpo del proceso que matchea la SL y, al anidar, también desde los sub-procesos.

Cadena de SLs

El input del jugador se parte en **una o varias SLs encadenadas**, separadas por conectores (y, comas...). Cada SL se procesa secuencialmente por el ciclo de tablas.

Si una SL contiene un bloque entrecomillado, ese bloque queda reservado y **no se parsea** hasta que la tabla **npc** lo procesa (ver anidamiento más abajo).

Ejemplos simples

- INPUT: `coger hacha`
- SL1: `verb: coger, noun: hacha, item: hacha`
- INPUT: `examinar troll`
- SL1: `verb: examinar, noun: troll, npc: troll`
- INPUT: `ex cueva`
- SL1: `verb: examinar, noun: cueva, npc: _, item: _`
- INPUT: `salidas`
- SL1: `verb: _, noun: salidas`

Ejemplos compuestos o anidados

- INPUT: decir hobbit "coge el hacha y corta leña" y coger leña
- SL1: verb: decir, noun: hobbit, npc: hobbit
 - SUBSL1: verb: coger, noun: hacha, item: hacha
 - SUBSL2: verb: cortar, noun: leña, item: leña
- SL2: verb: coger, noun: leña, item: leña

El último ITEM y el último NPC resueltos en la cadena se arrastran automáticamente a las SLs siguientes que no resuelvan uno propio.

- INPUT: coger la espada rápidamente y atacar al troll
- SL1: verb: coger, noun: espada, adverb: rápidamente, item: espada
- SL2: verb: atacar, noun: troll, npc: troll, item: espada ← item arrastrado de SL1
- INPUT: coger rápidamente la espada y atacar al troll
- SL1: verb: coger, noun: espada, adverb: rápidamente, item: espada
- SL2: verb: atacar, noun: troll, npc: troll, item: espada ← mismo resultado, distinto orden
- INPUT: sonreir al troll y bailar
- SL1: verb: sonreir, noun: troll, npc: troll
- SL2: verb: bailar, noun: _, npc: troll ← npc arrastrado de SL1

Un ejemplo más complicado:

- INPUT: coger rápidamente la espada, perseguir al troll y matarlo
- SL1: verb: coger, noun: espada, adverb: rápidamente, item: espada
- SL2: verb: perseguir, noun: troll, npc: troll, item: espada
- SL3: verb: atacar, noun: _, npc: troll, item: espada ← arrastra ambos de SL2

matarlo → 5 primeras letras: matar (sinónimo de atacar); el clítico lo queda descartado.

Realmente el programa tendría que estar preparado para resolver esto, veamos SL1 entra por item y el jugador coge la espada:

```
// tabla item
```

```
coger *:
  ISHERE ITEM
  GET ITEM
  OK
```

Entonces SL2 entra por npc y el jugador persigue al troll:

```
// tabla npc
```

```
perseguir troll:
  ISHERE NPC
  PRINT "¡Corres tras el troll que sale huyendo despavorido!"
  END
```

SL3 arrastra el último item y el último npc. Entra por item pero no hay entrada para atacar espada, así que sigue por npc y ataca al troll con la espada:

```
// tabla npc
```

```
atacar troll:
  ISHERE NPC
  EQ ITEM espada
  ISCARRIED ITEM
```

```
PRINT "¡Atacas al troll con la espada pero no llegas a herirle! El troll desaparece en el bosque."
DONE
```

El parser hace el trabajo

El parser hace el trabajo duro y se encarga de construir las SLs a partir del input del jugador. Sin embargo el programador tiene que tener en cuenta que debe introducir los suficientes datos en el fuente, los verbos, adverbios, sustantivos, sinónimos, conjunciones, etc. para que el parser pueda construir las SLs correctamente.

El parser solo tiene en cuenta las **5 primeras letras** de cada palabra del input. **examinar** y **exami** son equivalentes para el parser; el vocabulario debe definirse teniendo esto en cuenta para evitar colisiones. Este truncado también descarta de forma natural los clíticos verbales: **matarlo** se lee como **matar**, **cogela** como **coger**, etc.

Aún así, el parser no es infalible y puede fallar en algunos casos, pero es bastante avanzado y si el programador aprende como funciona, será un gran aliado.

Procesos

Un proceso es una entrada en una tabla de procesos que se ejecuta cuando se cumple la condición de entrada. Las condiciones de entrada son siempre **dos huecos**:

Tabla	Hueco 1	Hueco 2	Ejemplos
init	_	_	_ _
location	IN, OUT o *	localidad o *	IN celda, OUT *, * *
turn	EVERY o TIMEOUT	número entero positivo	EVERY 2, TIMEOUT 30
item	verbo, * o _	noun de item, * o _	coger denario, coger *, * *
npc	verbo, * o _	noun de NPC, * o _	decir elfo, buscar *
response	verbo, * o _	noun, * o _	examinar playa, _ *, * *
cron	unidad	número, o "HH:MM:SS" para AT	MINUTES 2, HOURS 1, AT "10:30:00"

Comodín y palabra vacía

- * — **comodín**, casa con cualquier label en esa posición.
- _ — **NullWord**, indica que esa posición de la SL debe estar vacía para encajar.

Ambos se admiten en **init**, **location**, **item**, **npc** y **response**. **No** se admiten en **turn** ni **cron**, cuyas cabeceras codifican un disparador, no un patrón de matching.

Ejemplos

Por ejemplo, en la **tabla de item**:

```
// tabla item
```

```
* *:
    NOT ISHERE ITEM
    PRINT item_not_here, ITEM
    DONE

coger *:
    ISCARRIED ITEM
    PRINT already_carried, ITEM
    NOOK

coger *:
    ISHERE vigilante
    PRINT vigilante-here
    NOOK
```

```
// A partir de aquí sabemos que el objeto está y el vigilante no.
```

```
coger *:  
    ISOVERWEIGHT ITEM  
    PRINT overweight, ITEM  
    NOOK
```

```
coger *:  
    GET ITEM  
    PRINT get, ITEM  
    OK
```

```
examinar *:  
    PRINT ITEM.description  
    DONE
```

Tabla de npc:

```
// tabla npc
```

```
* *:  
    NOT ISHERE NPC  
    PRINT npc_not_here, NPC  
    DONE
```

```
buscar *:  
    ISHERE NPC  
    PRINT "He encontrado a {1}.", NPC  
    DONE
```

```
buscar *:  
    PRINT "No encuentro a {1}.", NPC  
    DONE
```

```
examinar *:  
    PRINT NPC.description  
    DONE
```

Nota: Los mensajes no predefinidos en la sección de mensajes, crean un nuevo mensaje en la base de datos, con una etiqueta única generada automáticamente. Se busca si el mensaje ya existe para evitar duplicados. Esto rompe la internacionalización (por definir), y se recomienda no hacerlo como regla general, pero aquí los usamos para mayor claridad.

Cabeceras según la tabla

Como se definen las cabeceras puede tener pequeños cambios según la tabla.

Cabeceras en la tabla init Todas las cabeceras han de ser `_ _`, ya que todos los procesos se ejecutan en el arranque (o reinicio) y no hay SL que casar. Para agrupar procesos visualmente sólo se admiten comentarios.

Cabeceras en la tabla location El primer hueco siempre es el evento (`IN|OUT|*`), el segundo es la localidad o `*`. El proceso `IN` se ejecuta al entrar en esa localidad y el `OUT` se ejecuta al salir.

Por ejemplo:

```
IN *  
    // Se ejecuta al entrar en cualquier localidad  
OUT *
```

```

// Se ejecuta al salir de cualquier localidad
IN celda
// Se ejecuta al entrar en la celda
OUT celda
// Se ejecuta al salir de la celda
* *
// Se ejecuta al entrar o salir de cualquier localidad
* celda
// Al entrar o salir de la celda

```

Cabeceras en la tabla turn Dos formas de cabecera:

Cabecera	Descripción
EVERY n	El proceso se ejecuta cada n turnos.
TIMEOUT n	El proceso se ejecuta si el jugador no escribe en n segundos. El turno se da por consumido automáticamente.

La tabla **turn** se ejecuta **una sola vez por SL principal**; no se re-ejecuta para las SUBSLs del bloque entrecomillado. Al consumirse un **TIMEOUT** se ejecuta este proceso y cualquier otro **EVERY** compatible con el nuevo número de turnos, salvo que un terminador corte el ciclo antes, pero no hay **SL** disponible ni *binding* alguno.

Cabeceras en la tabla item Se entra en esta tabla **sólo si la SL contiene un item** (el **noun** resuelve a un **item** declarado). Las cabeceras casan contra **verb** y **noun** de item, su etiqueta en realidad. El binding **ITEM** está vivo dentro de los procesos.

Cabeceras en la tabla npc Se entra en esta tabla **sólo si la SL contiene un NPC** (el **noun** resuelve a un **character** no humano declarado). Las cabeceras casan contra **verb** y **noun** de NPC (su etiqueta o ID). El binding **NPC** está vivo dentro de los procesos.

Cabeceras en la tabla response Tabla general, se ejecuta tras **turn**, **item** y **npc**. Admite ***** y **_** en cualquiera de los dos huecos.

encender linterna:

```

SET linterna.on true
PRINT "Has encendido la linterna."
OK

```

guardar partida:

```

INPUT filename "Nombre de la partida: "
NOT IEMPTY filename
SAVE filename
OK

```

* insulto:

```

PRINT "{1} lo serás tú.", SL.noun
QUIT

```

- *:

```

PRINT "Cualquier nombre sin verbo."
DONE

```

* _:

```

PRINT "Cualquier verbo sin nombre."

```

DONE

```
* *:
  PRINT "No te entiendo."
  DONE
```

Cabeceras en la tabla cron Los procesos se ejecutan por intervalos. El primer hueco es la unidad, el segundo el valor; con AT el segundo es una cadena con formato "HH:MM:SS".

Unidades:

- HOURS — cada n horas (reloj de pared).
- MINUTES — cada n minutos.
- SECONDS — cada n segundos.
- AT — a una hora del día.

Los procesos cron son bloqueantes: si uno está en ejecución, el resto (cron u otra tabla) espera.

Fallo de condición en un proceso

Cuando una condición no se cumple, el proceso se corta automáticamente y se intenta ejecutar el siguiente que encaje.

Anidamiento de sub-procesos

Las tablas **response**, **item** y **npc** admiten **una capa** de anidamiento: dentro de un proceso pueden definirse sub-procesos con sus propias cabeceras. No se permite anidamiento en **init**, **location**, **turn** ni **cron**, ya que simplemente no tenemos *Sentencia Lógica* disponible.

TABLE npc

```
decir elfo:
  dar *:
    ISCARRIED ITEM NPC
    MOVE ITEM human
    PRINT "Toma _.", ITEM
    OK

  dar *:
    PRINT "No tengo _.", ITEM
    NOOK

* *:
  PRINT "El elfo no sabe responder a eso."
  NOOK
```

Reglas:

- **Una sola capa:** sin sub-sub-procesos.
- **Bindings heredados:** lo que armó el outer (NPC, ITEM, etc.) es visible desde los sub-procesos.
- **Preludio opcional:** el outer puede tener conductos antes de los sub-procesos. Esos conductos se ejecutan una vez al entrar al outer (y no se repiten en las re-entradas al mismo outer por sub-SLs distintas, ver más abajo).
- **Cierre implícito del outer:** termina al aparecer otra cabecera top-level o se termina la tabla.
- Los terminadores dentro de sub-procesos tienen la misma semántica que en el resto del lenguaje: **END** termina la tabla **npc** y continúa con **response**; **DONE/OK** terminan el ciclo y re-entran con la siguiente SUBSL o SL; **NOOK** descarta todo.

Hablar con NPCs: comillas y sub-SLs

Para dirigirse a un NPC el jugador **escribe la frase entre comillas**:

`decir elfo "dame la llave"`

El parser produce una SL principal con el contenido entrecomillado **sin parsear**:

`SL = (verbo=decir, nombre=elfo, NPC=elfo, contenido="dame la llave")`

Cuando el outer `decir elfo`: matchea en la tabla `npc`:

1. Se ejecuta el preludio del outer (si lo hay).
2. El parser extrae la **primera sub-SL** del contenido.
3. Los sub-procesos del outer se prueban contra esa sub-SL.
4. Si quedan más sub-SLs en el contenido, se **re-entra en el ciclo de tablas** con la misma SL outer y la siguiente SUBSL. La tabla `turn` no se re-ejecuta. El preludio del outer no se vuelve a ejecutar.

Ejemplo:

`decir elfo "coge la llave y abre la puerta"`

Expansión efectiva:

- SL principal: `decir elfo + contenido`.
- Sub-SL 1: `coger llave`.
- Sub-SL 2: `abrir puerta`.

Paso a paso:

1. Tabla `npc` matchea `decir elfo`:. Preludio ejecutado.
2. Sub-SL `coger llave` despachada contra los sub-procesos del outer.
3. Ciclo termina (sin `turn`). Quedan SUBSLs → re-entrada al ciclo con la misma SL `decir elfo` y SUBSL `abrir puerta`. Preludio no se repite.

SLs fuera de comillas

Lo que sigue al bloque entrecomillado son **SLs principales normales**, no dirigidas al NPC:

`decir elfo "dame la llave" y soltar bolsa, quitar abrigo`

- SL1: verb: `decir`, noun: `elfo`, npc: `elfo` → tabla `npc`
 - SUBSL1: verb: `dar`, noun: `llave`, npc: `elfo`, item: `llave` → tabla `npc submatch`
- SL2: verb: `soltar`, noun: `bolsa`, item: `bolsa` → tabla `item`
- SL3: verb: `quitar`, noun: `abrigo`, item: `abrigo` → tabla `item`

Catálogo de conductos

Condiciones

Negables con NOT condition.

Conducto	Parámetros	Descripción
NOOP	—	No hace nada
NOT	condition	Niega la condición siguiente
EQ	a b	Verdadero si <code>a == b</code>
GT	a b	Verdadero si <code>a > b</code>
LT	a b	Verdadero si <code>a < b</code>
GTE	a b	Verdadero si <code>a >= b</code>
LTE	a b	Verdadero si <code>a <= b</code>
ZERO	expr	Verdadero si <code>expr</code> es cero
CHANCE	n	Verdadero el n% de las veces (aleatorio)
ISHERE	entity	La entidad está en la misma ubicación que el jugador
ISAT	entity loc	La entidad está en la ubicación <code>loc</code>

Condacto	Parámetros	Descripción
ISCARRIED	item [who]	item está en el inventario de who (default: human)
ISWORN	item [who]	item está puesto por who (default: human)
ISPRESENT	entity	La entidad es ISHERE, ISCARRIED o está en un contenedor abierto
ISINSIDE	item item_container	item está dentro de item_container
ISEMPTY	container\ var	El contenedor está vacío, o var es cero/vacío
ISOVERWEIGHT	item [who]	Añadir item a who excedería el límite de peso (default: human)

Acciones

Inventario

Condacto	Parámetros	Descripción
GET	item [who]	Recoge item; actor: who (default: human)
DROP	item [who]	Deja item en la ubicación actual; actor: who
WEAR	item [who]	Equipa item como puesto; actor: who
UNWEAR	item [who]	Desequipa item al inventario; actor: who
PUTIN	item container	Mete item dentro de container
TAKEOUT	item container	Saca item de container
GIVE	item who	Mueve item del inventario del jugador al de who

Mundo

Condacto	Parámetros	Descripción
MOVE	entity loc	Mueve la entidad a loc
CREATE	item [loc]	Crea item en loc (default: ubicación actual)
DESTROY	item	Elimina item del juego hasta que CREATE lo recree
SWAP	entity entity	Intercambia la posición de dos entidades del mismo tipo
GOTO	loc	Mueve al jugador a loc (dispara eventos de localización)

Variables

Condacto	Parámetros	Descripción
SET	var value	Asigna value a var o a una propiedad
INC	var [n]	Incrementa var en n (default: 1)
DEC	var [n]	Decrementa var en n (default: 1)
RANDOM	var min max	Asigna a var un valor aleatorio en [min, max]

Terminal

Condacto	Parámetros	Descripción
WINDOW	id	Abre o selecciona una ventana
AT	row col	Posiciona el cursor
PRINT	msg\ prop	Imprime un mensaje o el valor de una propiedad
CURSOR	on\ off	Muestra u oculta el cursor
CLEAR	[window]	Limpia la pantalla o una ventana
BG	color	Establece el color de fondo
FG	color	Establece el color de primer plano

Media

Condacto	Parámetros	Descripción
PLAY	<code>blob</code>	Reproduce audio
PICTURE	<code>blob</code>	Muestra una imagen

Input y tiempo

Condacto	Parámetros	Descripción
INPUT	<code>var prompt</code>	Lee texto del jugador en <code>var</code>
WAIT	<code>ms</code>	Pausa la ejecución <code>ms</code> milisegundos
ANYKEY	—	Pausa hasta que el jugador pulse una tecla

Persistencia

Condacto	Parámetros	Descripción
SAVE	<code>[slot]</code>	Guarda el estado del juego
LOAD	<code>[slot]</code>	Carga el estado del juego

Control del juego

Condacto	Parámetros	Descripción
PARSE	<code>str</code>	Re-parsea <code>str</code> como si el jugador lo hubiera escrito
ENGAGE	<code>npc</code>	Inicia el dispatch de diálogo con <code>npc</code>
RESTART	—	Reinicia la aventura desde el principio
QUIT	—	Sale del juego

Control de proceso

Condacto	Parámetros	Descripción
DONE	—	Termina el ciclo de tablas; continúa con la siguiente SUBSL o SL
OK	—	Imprime <code>ok</code> ; termina el ciclo; continúa con siguiente SUBSL o SL
END	—	Termina la tabla actual; continúa con la siguiente del ciclo
NOOK	—	Imprime <code>nook</code> ; descarta todas las SLs y SUBSLs restantes